

The Pantheon Architecture Suite

Calypso16/32

Team members:
Mariam Abdullah
Sara Abdelaty
Manar Abdo
Mohamed Osama
Adel Osama

Calypso16/32: Architectural Design

1. Architectural Overview

Calypso16/32 is a custom 64-bit variable-length RISC processor. It supports both 16-bit compressed and 32-bit standard instruction formats, allowing a mix of code density and expressive power within a single instruction stream.

1.1 Processor Specifications

Property	Value
Architecture Name	Calypso16/32
Data Width (XLEN)	64 bits
Register File	32 general-purpose registers (x0 – x31)
Register Width	64 bits each (8 bytes)
x0 Behavior	Hardwired to zero. Writes are silently discarded. Reads always return 0.
Instruction Length	Variable: 16-bit (compressed) or 32-bit (standard)
Byte Order	Big-Endian (most significant byte at lowest address)
Memory Model	Byte-addressable, 64 KB address space (addresses 0x0000 – 0xFFFF)
Memory Alignment	Misaligned accesses are allowed (no alignment exceptions)
Word Size for Load/Store	8 bytes (64-bit doubleword)
Encoding Formats	4 formats: CS, CI (16-bit) and R, I (32-bit)
Supported Extensions	Base Integer (I) + Multiplication/Division (M)

1.2 Register File

The processor contains 32 registers, each 64 bits wide:

Register	Description
x0	Hardwired to 0 (constant zero)
x1	General-purpose (commonly used as return address)
x2 - x31	General-purpose registers

- Each register: 64 bits = 8 bytes
- Total register file: 32 x 8 = 256 bytes
- x0 Hardwiring Rule: Any instruction that writes to x0 has no effect. Any instruction that reads from x0 always receives the value 0. This provides a convenient constant zero for arithmetic and comparison operations.

1.3 Memory Layout

The 64 KB memory is organized into a data region and a program region to prevent code from overwriting itself during Store operations:

Address	Contents
-----	-----
0x0000	+-----+
	DATA REGION
	(4,096 bytes available)
	Used by LW/SW instructions to
	load and store 64-bit values.
0x0FFF	+-----+
0x1000	+-----+
	PROGRAM REGION
	(61,440 bytes available)
	Instructions are fetched from
	this region. PC starts at 0x1000.
0xFFFF	+-----+

2. Instruction Length Detection

2.1 The Problem

Calypso16/32 supports two instruction sizes within the same program: 16-bit (compressed) and 32-bit (standard). The fetch unit must determine the length of each instruction before it can read the correct number of bytes and advance the PC properly.

2.2 The Solution: MSB Prefix Bit

The most significant bit (bit 15) of the first halfword read from memory determines the instruction size:

bit[15] Value	Instruction Type	Action
0	16-bit instruction	Read 2 bytes only
1	32-bit instruction	Read 4 bytes total

This design ensures that the fetch unit can determine instruction length by examining a single bit in the very first byte, before reading any additional bytes.

2.3 Why This Works with Big-Endian

In Big-Endian byte order, the most significant byte is stored at the lowest address. This means bit[15] of the instruction is located in bit[7] (the MSB) of the very first byte in memory:

Instruction Type	Memory Layout (MSB First)
16-bit instruction at address A	memory[A] = bits [15:8] <-- bit[15] is the MSB of this byte

	memory[A+1] = bits [7:0]
32-bit instruction at address A	memory[A] = bits [31:24] <-- bit[31] is the MSB of this byte (matches bit[15] of halfword) memory[A+1] = bits [23:16] memory[A+2] = bits [15:8] memory[A+3] = bits [7:0]

Since Big-Endian places the most significant byte first, the CPU only needs to read one byte and check its top bit to know the instruction size.

2.4 Fetch Process — Step by Step

Step 1: Read 2 bytes from memory[PC] in Big-Endian to get the first halfword. Then evaluate bit[15]:

Condition	Outcome	Next Action	PC Advancement
bit[15] = 0	16-bit instruction	Done! Instruction = halfword	PC += 2
bit[15] = 1	32-bit instruction	Read 2 MORE bytes from memory[PC+2], combine into 32-bit	PC += 4

3. Program Counter (PC) Update Rules

The Program Counter (PC) holds the memory address of the current instruction. After each instruction executes, the PC must be updated to point to the next instruction.

3.1 Standard PC Advancement

Since Calypso16/32 has variable-length instructions, the PC advances by different amounts depending on the instruction size:

Instruction Size	PC Update
After a 16-bit instruction	PC = PC + 2
After a 32-bit instruction	PC = PC + 4

3.2 Branch (BEQ) PC Update

BEQ compares two registers and conditionally changes the PC. Note: BEQ is always a 32-bit instruction.

Condition	Outcome	PC Update
If rs1 == rs2	Branch taken	PC = PC + sign_extend(offset)
If rs1 != rs2	Branch not taken	PC = PC + 4

3.3 Jump (JAL) PC Update

JAL unconditionally jumps and saves the return address. Note: JAL is always a 32-bit instruction, so the return address is always PC + 4.

Step	Action	Description
Step 1	rd = PC + 4	Save address of next instruction
Step 2	PC = PC + sign_extend(offset)	Jump to target

3.4 Complete PC Update Summary

Instruction Type	Condition	PC Update	Return Address
Any 16-bit (CS/CI)	Always	PC = PC + 2	N/A
Any 32-bit ALU (R/I)	Always	PC = PC + 4	N/A
LW / SW (32-bit)	Always	PC = PC + 4	N/A
BEQ (32-bit)	Taken	PC = PC + sign_extend(offset)	N/A
BEQ (32-bit)	Not taken	PC = PC + 4	N/A
JAL (32-bit)	Always	PC = PC + sign_extend(offset)	rd = PC + 4

3.6 Halt Mechanism

The emulator detects a halt condition when it encounters JAL x0, 0:

- This saves the return address to x0 (which is discarded since x0 is hardwired to 0)
- This jumps to PC + 0 (which is the same instruction, an infinite loop)
- The emulator recognizes this pattern and stops execution instead of looping forever

4. Big-Endian Byte Ordering

4.1 Instruction Storage

All instructions are stored in memory with the most significant byte at the lowest address:

Address	Byte Value	Description (32-bit instruction 0xC0400014)
0x1000	0xC0	bits 31-24, most significant
0x1001	0x40	bits 23-16
0x1002	0x00	bits 15-8
0x1003	0x14	bits 7-0, least significant

Address	Byte Value	Description (16-bit instruction 0x0147)
0x1008	0x01	bits 15-8, most significant
0x1009	0x47	bits 7-0, least significant

4.2 Data Storage (64-bit Words)

LW and SW operate on 8-byte (64-bit) doublewords, also stored Big-Endian. Example storing value 30 (0x000000000000001E) to address 0x0000:

Address	Byte Value	Description
0x0000	0x00	bits 63-56, MSB
0x0001	0x00	bits 55-48
0x0002	0x00	bits 47-40
0x0003	0x00	bits 39-32
0x0004	0x00	bits 31-24
0x0005	0x00	bits 23-16
0x0006	0x00	bits 15-8
0x0007	0x1E	bits 7-0, LSB

5. Compressed Instruction Formats (16-bit)

Calypso16/32 provides two 16-bit compressed formats. These are shorter encodings for common operations, using destructive addressing to save space.

5.1 What is Destructive Addressing?

In a 16-bit instruction, there are only 16 bits to work with. With 32 registers requiring 5-bit specifiers, fitting three register fields would require 15 bits, leaving only 1 bit for the opcode, which is not enough.

The solution is destructive addressing: the destination register (rd) is also used as the first source register (rs1). This means the original value in rd is overwritten (destroyed) by the result.

Non-Destructive (32-bit, 3 registers)	Destructive (16-bit, 2 registers)
ADD x3, x1, x2	C.ADD x3, x2
$x3 = x1 + x2$	$x3 = x3 + x2$
x1 is preserved	x3's old value is lost
Bit budget: 5 (rd) + 5 (rs1) + 5 (rs2) = 15 bits + opcode = 32 bits needed	Bit budget: 5 (rd/rs1) + 5 (rs2) = 10 bits + opcode = Fits in 16 bits

5.2 CS Format (Compact-Short: Register-Register)

Used for compact register-register ALU operations. Uses destructive addressing (rd = rd OP rs2).

Assembly syntax: C.ADD rd, rs2

Bits [15:14]	Bits [13:10]	Bits [9:5]	Bits [4:0]
00	funct4	rd / rs1	rs2
prefix = CS format	operation selector (4 bits)	destination AND first source (5 bits)	second source register (5 bits)

Supported operations (Base I-Extension):

funct4	Binary	Instruction	Operation
--------	--------	-------------	-----------

0	0000	C.ADD	$rd = rd + rs2$
1	0001	C.SUB	$rd = rd - rs2$
2	0010	C.AND	$rd = rd \& rs2$
3	0011	C.OR	$rd = rd rs2$
4	0100	C.SLT	$rd = (\text{signed}(rd) < \text{signed}(rs2)) ? 1 : 0$

Supported operations (M-Extension):

funct4	Binary	Instruction	Operation
5	0101	C.MUL	$rd = (rd \times rs2)[63:0]$
6	0110	C.DIV	$rd = (rs2 == 0) ? -1 : \text{signed}(rd) / \text{signed}(rs2)$
7	0111	C.REM	$rd = (rs2 == 0) ? rd : \text{signed}(rd) \% \text{signed}(rs2)$

5.3 CI Format (Compact-Immediate: Register-Immediate)

Used for compact register-immediate ALU operations with a 6-bit sign-extended immediate.

Assembly syntax: C.ADDI rd, imm (meaning: $rd = rd \text{ OP } \text{sign_extend}(\text{imm})$)

Bits [15:14]	Bits [13:11]	Bits [10:6]	Bits [5:0]
01	funct3	rd / rs1	imm6
prefix = CI format	operation selector (3 bits)	destination AND first source (5 bits)	6-bit immediate (sign-extended to 64 bits)

Supported operations (Base I-Extension):

funct3	Binary	Mnemonic	Semantic
0	000	C.ADDI	$rd = rd + \text{sign_extend}(\text{imm6})$
1	001	C.ANDI	$rd = rd \& \text{sign_extend}(\text{imm6})$
2	010	C.ORI	$rd = rd \text{sign_extend}(\text{imm6})$
3	011	C.SLTI	$rd = (\text{signed}(rd) < \text{sign_extend}(\text{imm6})) ? 1 : 0$

6. R Format (Register, 32-bit)

Full 3-operand register-register operations with separate rd, rs1, and rs2.

Assembly syntax: ADD rd, rs1, rs2 (meaning: $rd = rs1 \text{ OP } rs2$)

Bits [31:30]	Bits [29:26]	Bits [25:21]	Bits [20:16]	Bits [15:11]	Bits [10:0]
10	funct4	rd	rs1	rs2	00000000000
prefix = R format	operation selector (4 bits)	destination register (5 bits)	first source register (5 bits)	second source register (5 bits)	reserved (11 bits)

Supported operations (Base I-Extension):

funct4	Binary	Mnemonic	Semantic
0	0000	ADD	$rd = rs1 + rs2$
1	0001	SUB	$rd = rs1 - rs2$
2	0010	AND	$rd = rs1 \& rs2$
3	0011	OR	$rd = rs1 rs2$
4	0100	SLT	$rd = (\text{signed}(rs1) < \text{signed}(rs2)) ? 1 : 0$

Supported operations (M-Extension):

funct4	Binary	Mnemonic	Semantic
5	0101	MUL	$rd = (rs1 \times rs2)[63:0]$
6	0110	DIV	$rd = (rs2 \neq 0) ? -1 : \text{signed}(rs1) / \text{signed}(rs2)$
7	0111	REM	$rd = (rs2 \neq 0) ? rs1 : \text{signed}(rs1) \% \text{signed}(rs2)$

7. I Format (Immediate, 32-bit)

Operations with a 16-bit immediate. Used for immediate ALU ops, loads, stores, branches, and jumps.

Assembly syntax: ADDI rd, rs1, imm

Bits [31:30]	Bits [29:26]	Bits [25:21]	Bits [20:16]	Bits [15:0]
11	funct4	rd / rs2	rs1	imm16
prefix = I format	operation selector (4 bits)	destination (or source for SW/BEQ) (5 bits)	source / base-address register (5 bits)	16-bit immediate (sign-extended to 64 bits)

Supported operations (Base I-Extension):

funct4	Binary	Mnemonic	Field Usage	Semantic
0	0000	ADDI	rd, rs1, imm	$rd = rs1 + \text{sign_extend}(\text{imm16})$

1	0001	ANDI	rd, rs1, imm	rd = rs1 & sign_extend(imm16)
2	0010	ORI	rd, rs1, imm	rd = rs1 sign_extend(imm16)
3	0011	SLTI	rd, rs1, imm	rd = (signed(rs1) < sign_extend(imm16)) ? 1 : 0
4	0100	LW	rd, rs1, imm	rd = Mem[rs1 + sign_extend(imm16)]
5	0101	SW	rs2, rs1, imm	Mem[rs1 + sign_extend(imm16)] = rs2
6	0110	BEQ	rs2, rs1, imm	if (rs1 == rs2) PC += sign_extend(imm16); else PC += 4
7	0111	JAL	rd, imm	rd = PC + 4; PC += sign_extend(imm16)

8. OOP Class Architecture

The emulator is structured as five cooperating C++ classes. Each class owns a single well-defined responsibility and communicates through references, eliminating global mutable state.

Class	File(s)	Responsibility
RegisterFile	cpu_state.h	32 x 64-bit GPRs; x0 hardwired to zero
CPUState	cpu_state.h	Owns RegisterFile, memory[], PC, running
Memory	memory.h / .cpp	64-bit Big-Endian load/store; bounds checks
Decoder	decode.h / .cpp	Static decode() — CS, CI, R, I formats
Fetcher	fetch.h / .cpp	Reads 16-bit or 32-bit word from cpu_.memory
Executor	execute.h / .cpp	Executes one Decoded instruction

9. Fetch Stage Rules

The Fetcher class reads one instruction word from cpu_.memory[] at the current PC using Big-Endian byte order.

Instruction-size detection

- Read two bytes at PC as a Big-Endian 16-bit halfword (first_halfword).
- If bit[15] of first_halfword == 0: the instruction is 16 bits (CS or CI). inst_size = 2.

- If bit[15] of first_halfword == 1: read two more bytes at PC+2 and combine into a 32-bit word. inst_size = 4.
- Return false (fetch failure) if PC + inst_size would exceed 0xFFFF.

10. Execute Stage Rules

10.1 ALU operations (CS, CI, R, I funct4 0-3)

- CS / R formats: two-register operands (rs1_val = rf.read(rs1), rs2_val = rf.read(rs2)).
- CI / I formats (funct4 ≤ 3): one register + sign-extended immediate.
- Result written to rd via rf.write(rd, result). x0 writes are silently discarded.

10.2 Memory operations (I funct4 4-5)

- LW: addr = rf.read(rs1) + sign_extend(imm16). Loads 8 bytes Big-Endian. Rejects accesses that exceed the 64 KiB boundary.
- SW: addr = rf.read(rs1) + sign_extend(imm16). Stores 8 bytes Big-Endian. Rejects accesses that would enter the program region (addr ≥ 0x1000) or exceed the boundary.

10.3 Verbose runtime trace (normal mode)

The normal (non-test) run prints a per-cycle trace that shows only the registers that changed, memory activity, and the PC transition. At the end a summary lists all non-zero registers.

Example trace output

```

CYCLE 3  [0x1004 -> 0x1008]  SW x1, 0(x0)
  Changed registers:
    (none)
  Store : MEM[0x0000] <- x1 = 10

CYCLE 4  [0x1008 -> 0x100C]  LW x3, 0(x0)
  Changed registers:
    x3 : 0 -> 10 (0x0000000000000000A)
  Load  : x3 <- MEM[0x0000] = 10

```

10.4 PC Update Rules

Instruction class	PC rule
Any 16-bit CS / CI	PC = PC + 2
Any 32-bit R / I ALU	PC = PC + 4
LW / SW	PC = PC + 4
BEQ taken	PC = PC + SignExt(imm16)
BEQ not taken	PC = PC + 4
JAL	rd = PC + 4; PC = PC + SignExt(imm16)
JAL x0, 0	Explicit HALT: running = false

10.5 Memory and Protection Rules

- Memory is byte-addressable from 0x0000 through 0xFFFF (64 KiB).

- Data is stored and loaded in Big-Endian order; the most-significant byte occupies the lowest address.
- LW/SW transfer 8 bytes and permit misaligned addresses as long as the entire 8-byte access remains inside the data region (0x0000–0x0FFF).
- Accesses that extend beyond 0xFFFF are rejected.
- Stores that would enter the program region (0x1000 and above) are rejected so that code cannot be overwritten.

10.6 Arithmetic and M-Extension Edge Cases

Operation	Required behavior	Implementation note
ADD/SUB	64-bit modular arithmetic	Unsigned storage; defined wrap-around.
SLT/SLTI	Signed comparison	Cast to int64_t at comparison time.
MUL	Signed product, lower XLEN bits	Result kept as lower 64 bits.
DIV	Signed quotient; div-by-zero → −1	Returns −1 for zero divisor.
REM	Signed remainder; rem-by-zero → rs1	Returns rs1 for zero divisor.

11. Mixed-Length Execution Demonstration

The built-in demonstration intentionally mixes 16-bit and 32-bit instructions so the runtime output makes variable-length operation obvious.

Cycle	PC	Instruction	Effect	Next PC
1	0x1000	C.ADDI x1, 5	x1 = 5	0x1002
2	0x1002	C.ADD x1, x1	x1 = 10	0x1004
3	0x1004	SW x1, 0(x0)	Data memory ← 10	0x1008
4	0x1008	LW x3, 0(x0)	x3 = 10	0x100C
5	0x100C	JAL x0, 0	HALT	0x100C

Final expected state

x1 = 10, x2 = 0, x3 = 10, PC = 0x100C, running = false. Data memory bytes 0x0000–0x0007 represent the 64-bit value 10 in Big-Endian order (00 00 00 00 00 00 00 0A).

12. Built-in Self-Test Suite

The self-test mode is invoked with the --self-test flag. Each test prints architectural context before the PASS/FAIL result: PC, raw encoding, assembly where applicable, decoded fields, expected behavior, actual state, register-file snapshot, and memory bytes when relevant. The suite ends with a machine-checkable summary.

12.1 Self-Test Matrix — Tests 1 to 22

#	Test name	Coverage demonstrated
---	-----------	-----------------------

01	CS decode	16-bit CS prefix and register extraction.
02	CI decode + 6-bit sign ext.	CI fields and signed imm6 (e.g. -3).
03	R decode	32-bit R prefix and register extraction.
04	I decode + 16-bit sign ext.	32-bit I fields and signed imm16 (e.g. -9).
05	CS disassembly	C.ADD text rendering.
06	CI disassembly	C.ADDI with negative immediate.
07	R disassembly	MUL text rendering.
08	I disassembly	LW text rendering.
09	x0 ignores writes	Write protection for x0.
10	x0 reads as zero	Read rule for x0.
11	ADD	Register-register addition.
12	SUB	Register-register subtraction.
13	AND	Bitwise AND.
14	OR	Bitwise OR.
15	SLT signed	Signed comparison.
16	ADDI with negative immediate	Immediate arithmetic and sign extension.
17	ANDI	Immediate AND.
18	ORI	Immediate OR.
19	SLTI signed	Signed immediate comparison.
20	CS ADD/SUB/AND/OR/SLT/MUL/DIV/REM	All eight CS operations.
21	C.ADDI immediate execution	Compressed immediate addition.
22	C.ANDI execution	Compressed immediate AND.

12.2 Self-Test Matrix — Tests 23 to 43

#	Test name	Coverage demonstrated
23	C.ORI execution	Compressed immediate OR.
24	C.SLTI execution	Compressed signed immediate comparison.
25	C.ADDI minimum signed immediate	Minimum 6-bit signed immediate.
26	MUL signed low 64 bits	Signed multiply, lower 64-bit result.
27	DIV signed	Signed quotient.
28	REM signed	Signed remainder.
29	DIV by zero → all ones	Division-by-zero rule.
30	REM by zero → rs1	Remainder-by-zero rule.
31	Misaligned SW/LW round trip	Allowed misaligned 64-bit data access.

32	Big-endian data storage	Byte ordering for 64-bit data.
33	Store crossing into program region	Protection of program region.
34	Out-of-bounds memory read rejected	Memory boundary check on read.
35	Out-of-bounds memory write rejected	Memory boundary check on write.
36	BEQ taken target	Taken branch target calculation.
37	BEQ not-taken fall-through	Sequential branch behavior.
38	JAL return address + target	Return address and target update.
39	Reserved CS funct detected	Reserved function code detection.
40	Illegal instruction halts emulator	Illegal execute path stops emulator.
41	Mixed-length fetch integration	Full mixed 16/32-bit pipeline.
42	Mixed-length program final state	Final register/PC/running state.
43	Final result Big-Endian in memory	Final data-memory encoding.

12.3 Verification Summary

Verification item	Expected / observed result
Build	Clean compilation: C++17, -Wall, -Wextra, -pedantic.
Full self-test	43 passed, 0 failed.
Mixed-length demo	5 instruction cycles, reaches explicit HALT.
Final demo state	x1=10, x3=10, PC=0x100C, running=false.
Final memory	0x0000–0x0007 = 00 00 00 00 00 00 00 0A.
Pipeline visibility	Fetch, Decode, Disassemble, Execute, PC update printed.

13. Build, Run, and Demonstration Commands

g++ direct build

```
g++ -std=c++17 -Wall -Wextra -pedantic *.cpp -o calypso
```

CMake build

```
cmake -B build
cmake --build build
```

Run — normal demo (shows changed registers only)

```
.\calypso.exe
```

Run — full verbose self-test suite

```
.\calypso.exe --self-test
```

Windows batch helper

run_tests.bat

14. Submission Readiness Checklist

- Architecture constants and architectural rules are documented.
- All four formats are documented: CS, CI, R, I.
- All required Base I + M instructions are listed with semantics.
- Fetch, Decode, Disassemble, Execute, and PC-update behavior are documented.
- Memory byte ordering, 64-bit LW/SW, misalignment, bounds, and store protection are documented.
- Runtime demo is self-explanatory: shows only changed registers per cycle.
- 43-test verification matrix is included.
- The completed software package preserves the original source organisation and adds fetch.h/.cpp and self_tests.h/.cpp for the OOP refactor.

15. Revision Record

Revision item	Added / completed
32-bit R format	Field layout, operation table, decode, disassembly, execution coverage.
32-bit I format	Field layout, operation table, decode, disassembly, memory/control-flow execution.
OOP class architecture	Five classes: RegisterFile, CPUState, Memory, Decoder, Fetcher, Executor.
Verbose normal run	Per-cycle output shows only changed registers, memory activity, and PC.
Verbose self-test	Readable architectural trace for every test plus PASS/FAIL status.
Memory protection	Program-region crossing stores rejected; out-of-bounds checks explicit.
Edge-case arithmetic	Signed SLT/DIV/REM, divide-by-zero rules, and 64-bit multiply covered.
Project support files	README, CMake, CMakeLists.txt, run_tests.bat.